Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №3
на тему
«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ. КРИПТОСИСТЕМА РАБИНА»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Краткие теоретические сведения 4
2.1 Ключи 4
2.2 Зашифрование 4
2.3 Расшифрование 4
2.4 Неоднозначность расшифрования 5
3 Выполнение работы 6
3.1 Структура программного средства 6
3.2 Генерация ключей 6
3.3 Зашифрование 6
3.4 Расшифрование 6
3.5 Интерфейс 7
3.6 Тестирование 7
Заключение 8
Список использованных источников 9
Приложение А (Обязательное) Листинг программного кода 10

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение криптосистемы Рабина и получение практических навыков построения асимметричных криптосистем.

В ходе работы требовалось реализовать программное средство шифрования и дешифрования текстовых файлов на основе криптосистемы Рабина. Для этого необходимо:

1 реализовать генерацию ключевой пары: двух простых чисел $p, q \equiv 3 \pmod 4$ и открытого модуля $n = pq$;

2 реализовать зашифрование сообщения возведением в квадрат по модулю n;

3 реализовать расшифрование через вычисление квадратных корней по модулю простых чисел и китайскую теорему об остатках;

4 устранить неоднозначность расшифрования, при которой у шифртекста существует четыре квадратных корня;

5 встроить криптосистему в терминальный интерфейс программы и проверить реализацию тестами.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Ключи

Криптосистема Рабина – асимметричная схема, стойкость которой опирается на сложность извлечения квадратных корней по модулю составного числа, эквивалентную факторизации n [1].

Выбираются два случайных простых числа p и q, удовлетворяющих условиям

$$p \equiv q \equiv 3 \pmod 4, \quad p \neq q. \tag{2.1}$$

Открытым ключом является $n = pq$, закрытым – пара (p, q).

2.2 Зашифрование

Сообщение m, представленное числом $0 \leq m < n$, шифруется по формуле

$$c = m^2 \bmod n. \tag{2.2}$$

Операция сводится к одному возведению в квадрат, поэтому зашифрование быстрее, чем в схеме RSA при той же длине ключа.

2.3 Расшифрование

По закрытому ключу вычисляются квадратные корни по модулю каждого простого числа:

$$m_p = c^{\frac{p+1}{4}} \bmod p, \quad m_q = c^{\frac{q+1}{4}} \bmod q. \tag{2.3}$$

Показатели целые, поскольку $p \equiv q \equiv 3 \pmod 4$.

Из расширенного алгоритма Евклида находятся коэффициенты y_p и y_q, для которых $y_p p + y_q q = 1$. Тогда по китайской теореме об остатках четыре корня по модулю $n = pq$ имеют вид

$$r = \left(m_p y_q q + m_q y_p p\right) \bmod n, \quad s = \left(m_p y_q q - m_q y_p p\right) \bmod n, \tag{2.4}$$

$$-r = n - r, \quad -s = n - s. \tag{2.5}$$

Одно из четырёх значений является исходным сообщением m, остальные три – посторонними корнями.

2.4 Неоднозначность расшифрования

Основной недостаток схемы состоит в том, что получатель не знает, какой из четырёх корней истинный. На практике неоднозначность устраняют избыточностью: к сообщению перед зашифрованием добавляют известную получателю структуру, и после расшифрования выбирается тот корень, который эту структуру содержит [2].

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Криптосистема реализована на языке Go и разделена на три уровня:

– *crypto/rabin/rabin.go* – примитив: генерация ключей, зашифрование, вычисление корней и восстановление сообщения;

– *cipher/rabin.go* – адаптер к интерфейсу *AsymCipher*, через который примитив использует интерфейс программы;

– *ui/asympane* – терминальная панель асимметричного шифрования.

Адаптер хранит открытый ключ как число n, а закрытый – как конкатенацию p и q с фиксированной шириной каждого простого числа. Панель предлагает три размера модуля: 512, 1024 и 2048 бит.

3.2 Генерация ключей

Функция *GenerateKeys* получает простые числа стандартным генератором *crypto/rand* и принимает только те, у которых второй бит равен единице, то есть $p \equiv 3 \pmod 4$ при совпадении чисел генерация повторяется. Модуль n и пара простых чисел образуют ключевую пару.

3.3 Зашифрование

Прямое применение формулы $c = m^2 \bmod n$ к тексту требует разбиения на блоки и решения проблемы неоднозначности. В реализации блок имеет длину на один байт меньше длины модуля, поэтому любой блок гарантированно меньше n.

В каждый блок перед возведением в квадрат добавляется избыточность: за полезными данными записывается маркер `0x80`, а последние восемь байт блока занимает метка `MZIRABIN`. Маркер отделяет полезные данные от нулевого заполнения, поэтому сообщения, оканчивающиеся нулевыми байтами, восстанавливаются корректно. Полезная нагрузка блока равна его длине минус девять байт.

Сообщение разбивается на блоки, каждый блок возводится в квадрат по модулю n и записывается в k байт, где k – длина модуля в байтах. Пустое сообщение также даёт один блок.

3.4 Расшифрование

Для каждого блока вычисляются четыре корня. Каждый корень интерпретируется как блок и проверяется: он должен заканчиваться меткой, а перед нулевым заполнением должен стоять маркер `0x80`. Избыточность отбрасывается, и остаётся полезная нагрузка.

Верный корень определяется однозначно: посторонние корни с подавляющей вероятностью не содержат корректной избыточности. Поэтому в отличие

от учебной схемы пользователю не нужно выбирать сообщение из четырёх вариантов – дешифратор возвращает один открытый текст. Если ни один корень не прошёл проверку, возвращается ошибка *ErrNoValidRoot*, что соответствует неверному ключу или повреждённому шифртексту. Также отвергаются шифртекст неправильной длины и блоки со значением не меньше n.

3.5 Интерфейс

Криптосистема встроена в терминальный интерфейс программы отдельной вкладкой *Rabin*. Клавиша *Ctrl+G* генерирует ключевую пару выбранного размера, закрытый ключ скрыт и показывается по *Ctrl+R*. Входные данные можно ввести вручную или выбрать файл, режим *Encrypt* или *Decrypt* переключается по *Enter*, результат выводится в шестнадцатеричном виде и может быть сохранён в файл. Открытый и закрытый ключи также задаются в шестнадцатеричном виде, что позволяет сохранять их между запусками.

3.6 Тестирование

Корректность проверена модульными тестами пакета *crypto/rabin* командой `go test ./internal/crypto/rabin`.

– *TestLabExampleRoots* воспроизводит пример с параметрами $p = 7$, $q = 11$, $n = 77$ и $m = 20$ [3]; получается $c = 15$, а корнями являются числа 64, 20, 13 и 57.

– *TestOnlyTrueRootCarriesRedundancy* проверяет, что из четырёх корней избыточность содержит ровно один – исходный блок.

– *TestRoundTrip* выполняет круговое шифрование и дешифрование на 512-битном модуле для пустого сообщения, короткой строки, текста из нескольких блоков, сообщения с нулевыми байтами в конце и случайных двоичных данных.

– *TestDecryptRejectsBadInput* проверяет отказ при обрезанном шифртексте и при закрытом ключе из одинаковых чисел.

Все тесты проходят.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована криптосистема Рабина: генерация ключей с простыми числами $p \equiv q \equiv 3 \pmod 4$, зашифрование возведением в квадрат по модулю n и расшифрование через корни по модулю простых чисел и китайскую теорему об остатках. Неоднозначность из четырёх корней устранена избыточностью в блоке, поэтому получатель восстанавливает единственный открытый текст без выбора варианта. Криптосистема встроена в терминальный интерфейс с поддержкой ручного ввода и файлов, корректность подтверждена модульными тестами.

Ограничением реализации является детерминированное зашифрование: одинаковая полезная нагрузка при одном ключе даёт одинаковый шифртекст, поэтому схема не скрывает повторов в данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Rabin, M.O. Digitalized Signatures and Public-Key Functions as Intractable as Factorization / M.O. Rabin. – 1979.

[2] Menezes, A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press, 1996. – 780 с.

[3] Лабораторные работы по дисциплине «Методы защиты информации» : методические указания. – 2024.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Криптосистема Рабина

```
package rabin

import (
  "bytes"
  "crypto/rand"
  "errors"
  "math/big"
)

var tag = []byte("MZIRABIN")

// Маркер конца данных: без него завершающие нулевые байты сообщения неотличимы
от паддинга.
const padMarker = 0x80

var (
  one = big.NewInt(1)
  two = big.NewInt(2)

  ErrKeyTooSmall = errors.New("modulus too small for redundancy block")
   ErrInvalidKey  = errors.New("private key must be two primes congruent to 3
mod 4")
  ErrBadCipher   = errors.New("ciphertext length or values do not match the key")
   ErrNoValidRoot = errors.New("no square root carries valid redundancy: wrong
key or corrupted ciphertext")
)

func GenerateKeys(bits int) (*big.Int, *big.Int, error) {
  p, err := randPrime3mod4(bits / 2)
  if err != nil {
    return nil, nil, err
  }
  for {
    q, err := randPrime3mod4(bits / 2)
    if err != nil {
      return nil, nil, err
    }
    if q.Cmp(p) != 0 {
      return p, q, nil
    }
  }
}

func randPrime3mod4(bits int) (*big.Int, error) {
  for {
    p, err := rand.Prime(rand.Reader, bits)
    if err != nil {
      return nil, err
    }
    if p.Bit(1) == 1 {
      return p, nil
    }
  }
}

func Encrypt(n *big.Int, msg []byte) ([]byte, error) {
```

```
  size, err := payloadLen(n)
  if err != nil {
    return nil, err
  }
  k := byteLen(n)
  pieces := chunk(msg, size)
  out := make([]byte, len(pieces)*k)

  // Блок и big.Int переиспользуются между блоками: на блок не приходится ни
одной аллокации.
  block := make([]byte, blockLen(n))
  m, c := new(big.Int), new(big.Int)
  for i, payload := range pieces {
    m.SetBytes(padInto(block, payload))
    c.Exp(m, two, n)
    c.FillBytes(out[i*k : (i+1)*k])
  }
  return out, nil
}

func Decrypt(p, q *big.Int, ct []byte) ([]byte, error) {
  if !validPrivate(p, q) {
    return nil, ErrInvalidKey
  }
  n := new(big.Int).Mul(p, q)
  if _, err := payloadLen(n); err != nil {
    return nil, err
  }
  k := byteLen(n)
  if len(ct) == 0 || len(ct)%k != 0 {
    return nil, ErrBadCipher
  }

  var plain []byte
  c := new(big.Int)
  for off := 0; off < len(ct); off += k {
    c.SetBytes(ct[off : off+k])
    if c.Cmp(n) >= 0 {
      return nil, ErrBadCipher
    }
    payload, err := recoverPayload(Roots(c, p, q), blockLen(n))
    if err != nil {
      return nil, err
    }
    plain = append(plain, payload...)
  }
  return plain, nil
}

func Roots(c, p, q *big.Int) []*big.Int {
  mp := sqrtModPrime(c, p)
  mq := sqrtModPrime(c, q)

   // yp*p + yq*q = 1: eq = yq*q ≡ 1 (mod p), ep = yp*p ≡ 1 (mod q) —
коэффициенты CRT.
  yp, yq := new(big.Int), new(big.Int)
  new(big.Int).GCD(yp, yq, p, q)
  n := new(big.Int).Mul(p, q)
  ep := new(big.Int).Mul(yp, p)
  eq := new(big.Int).Mul(yq, q)

  r := new(big.Int).Mul(mp, eq)
  r.Add(r, new(big.Int).Mul(mq, ep))
```

```
  r.Mod(r, n)

  s := new(big.Int).Mul(mp, eq)
  s.Sub(s, new(big.Int).Mul(mq, ep))
  s.Mod(s, n)

  return []*big.Int{r, negMod(r, n), s, negMod(s, n)}
}

func sqrtModPrime(c, p *big.Int) *big.Int {
  e := new(big.Int).Add(p, one)
  e.Rsh(e, 2)
  return new(big.Int).Exp(c, e, p)
}

func negMod(x, n *big.Int) *big.Int {
  return new(big.Int).Mod(new(big.Int).Neg(x), n)
}

func validPrivate(p, q *big.Int) bool {
  return p.Bit(1) == 1 && q.Bit(1) == 1 && p.Cmp(q) != 0
}

func byteLen(n *big.Int) int {
  return (n.BitLen() + 7) / 8
}

// Блок на один байт короче n, чтобы любой блок был меньше n.
func blockLen(n *big.Int) int {
  return byteLen(n) - 1
}

func payloadLen(n *big.Int) (int, error) {
  size := blockLen(n) - len(tag) - 1
  if size < 1 {
    return 0, ErrKeyTooSmall
  }
  return size, nil
}

// Всегда возвращает хотя бы один кусок: пустое сообщение тоже даёт блок.
func chunk(msg []byte, size int) [][]byte {
  var pieces [][]byte
  for len(msg) > size {
    pieces = append(pieces, msg[:size])
    msg = msg[size:]
  }
  return append(pieces, msg)
}

func pad(payload []byte, width int) []byte {
  return padInto(make([]byte, width), payload)
}

// padInto заполняет готовый блок целиком, чтобы буфер можно было
переиспользовать.
func padInto(block, payload []byte) []byte {
  clear(block) // иначе хвост предыдущего блока останется между маркером и tag
  copy(block, payload)
  block[len(payload)] = padMarker
  copy(block[len(block)-len(tag):], tag)
  return block
}
```

```
func recoverPayload(roots []*big.Int, width int) ([]byte, error) {
  for _, r := range roots {
    if payload, ok := blockPayload(r, width); ok {
      return payload, nil
    }
  }
  return nil, ErrNoValidRoot
}

func blockPayload(r *big.Int, width int) ([]byte, bool) {
  if r.BitLen() > 8*width {
    return nil, false
  }
  return unpad(r.FillBytes(make([]byte, width)))
}

func unpad(block []byte) ([]byte, bool) {
  if !bytes.HasSuffix(block, tag) {
    return nil, false
  }
  body := bytes.TrimRight(block[:len(block)-len(tag)], "\x00")
  if len(body) == 0 || body[len(body)-1] != padMarker {
    return nil, false
  }
  return body[:len(body)-1], true
}
```

Листинг А.2 – Адаптер к интерфейсу программы

```
package cipher

import (
  "errors"
  "math/big"

  "go-cipher/internal/crypto/rabin"
)

type Rabin struct{}

func NewRabin() Rabin {
  return Rabin{}
}

func (Rabin) Name() string {
  return "Rabin"
}

func (Rabin) KeyOptions() []KeyOption {
  return []KeyOption{
    {Label: "512-bit modulus", Bits: 512},
    {Label: "1024-bit modulus", Bits: 1024},
    {Label: "2048-bit modulus", Bits: 2048},
  }
}

// Публичный ключ — n, приватный — p||q с фиксированной шириной каждого простого.
func (Rabin) GenerateKeys(option KeyOption) (KeyPair, error) {
  p, q, err := rabin.GenerateKeys(option.Bits)
  if err != nil {
    return KeyPair{}, err
  }
```

```
  n := new(big.Int).Mul(p, q)
  primeBytes := option.Bits / 16
  return KeyPair{
    Public:  n.FillBytes(make([]byte, option.Bits/8)),
                Private:    append(p.FillBytes(make([]byte,    primeBytes)),
q.FillBytes(make([]byte, primeBytes))...),
  }, nil
}

func (Rabin) Encrypt(pub, msg []byte) ([]byte, error) {
  return rabin.Encrypt(new(big.Int).SetBytes(pub), msg)
}

func (Rabin) Decrypt(priv, cipherText []byte) ([][]byte, error) {
  if len(priv) == 0 || len(priv)%2 != 0 {
    return nil, errors.New("private key must be p||q of equal length")
  }
  half := len(priv) / 2
  p := new(big.Int).SetBytes(priv[:half])
  q := new(big.Int).SetBytes(priv[half:])

  plain, err := rabin.Decrypt(p, q, cipherText)
  if err != nil {
    return nil, err
  }
  return [][]byte{plain}, nil
}
```